IG AutoDM - Architecture Documentation

Instagram Comment → DM SaaS Platform

1. Overview

IG AutoDM is a multi-tenant SaaS platform where creators connect their Instagram accounts, configure trigger keywords, and automatically send public comment replies and private DMs via Meta's official Graph API. The system is built on free tiers across all infrastructure.

2. Frontend Architecture

2.1 Astro Static Site (apps/web)

- Static-site generator with `output: "static"`
- Landing page, login, and dashboard UI
- Uses Astro components (.astro) and layouts (Base.astro)
- Calls API via `PUBLIC_API_BASE` env variable (falls back to production URL)
- Auth: Supabase OAuth (Google/Facebook) + session management
- Pages: index.astro, login.astro, dashboard.astro, privacy.astro, data-deletion.astro, auth/callback.astro
- Global UI: Nav bar, footer, theming (dark/light with forest/coral/cream accents)

2.2 React + Vite + Tailwind (apps/web-react)

- Fully interactive dashboard SPA
- React 19, Vite 7, Tailwind CSS 4
- Pages: Landing.tsx, Login.tsx, Dashboard.tsx, AuthCallback.tsx, ConnectGuide.tsx, Onboarding.tsx, GetAccess.tsx
- UI components: Button, Badge, Header, Footer (in ui.tsx)
- State management: AuthContext (Supabase session)
- Communication: lib/api.ts wrapper calling `API_BASE` with Bearer tokens

3. Backend Architecture

3.1 Hono on Cloudflare Workers (apps/api/)

- Framework: Hono (lightweight TypeScript web framework)
- Runtime: Cloudflare Workers (`compatibility_date: "2025-07-18"`)
- ORM: Drizzle ORM with libSQL (Turso)
- Queue: Cloudflare Queues (`COMMENT_QUEUE`) for webhook event processing
- Cron: Two cron triggers (`0 2 * * *` + `* * * * *`) for polling/refresh
- Bindings: WEBBASE, API_BASE, OAUTH_BASE_URL, WEBHOOK_VERIFY_TOKEN, SUPABASE_URL, SUPABASE_ANON_KEY, META_APP_ID, META_APP_SECRET, ENCRYPTION_KEY, TURSO_DATABASE_URL, TURSO_AUTH_TOKEN, CLOUDINARY_URL

3.2 Key API Routes

Route	Description
GET /health	Health check
GET /me	Current user info
GET/POST /webhook/meta	Meta webhook verification + event processing
GET/POST /connect/meta	Instagram OAuth start + callback
GET /accounts	List connected IG accounts
POST /accounts/:igUserId/disconnect	Disconnect account + delete data
GET /stats	7-day reply/DM stats
GET /media?account=	List media with cached thumbnails + per-video configs
PUT /media-config	Create/update per-video override
PUT /accounts/:igUserId/defaults	Account-level defaults
GET/POST /byok	Bring Your Own Key management
POST /byok/check	Verify App ID + Secret format
GET /onboarding	Onboarding progress state
POST /access/request	Early access request form

4. Backend URLs & API Endpoints

Production URLs (from urls.txt)

- Dashboard: `https://ig-autodm.pages.dev/dashboard`
- Login: `https://ig-autodm.pages.dev/login`
- Privacy Policy: `https://ig-autodm.pages.dev/privacy`
- Data deletion: `https://ig-autodm.pages.dev/data-deletion`
- API health check: `https://ig-autodm-api.admingamergully.workers.dev/health`
- Webhook callback: `https://ig-autodm-api.admingamergully.workers.dev/webhook/meta`
- OAuth redirect: `https://ig-autodm-api.admingamergully.workers.dev/connect/meta/callback`
- Supabase site URL: `https://ig-autodm.pages.dev`

Meta Webhook Setup

- Callback URL: `https:///webhook/meta`
- Verify token: `local_verify_token`
- Subscribe to field: `comments`
- Oauth redirect URIs: `https://ig-autodm-api.admingamergully.workers.dev/connect/meta/callback`

5. Frontend-Backend Communication

Authentication Flow

1. Google OAuth → Supabase Auth session → dashboard
2. JWT token passed as `Authorization: Bearer` header on all API calls
3. Hono middleware `requireUser` verifies token with Supabase (`/auth/v1/user`)
4. Fallback: Demo mode when no token present

API Calls from Frontends

React (apps/web-react/src/lib/api.ts):

- Wraps all fetch calls to `${API_BASE}`
- Automatically adds `Content-Type: application/json` + `Authorization: Bearer`
- Named exports: `api.health()`, `api.me(token)`, `api.accounts(token)`, `api.media(token, account)`, `api.stats(token)`, etc.

Astro (apps/web/src/lib/supabase.ts):

- Same Supabase client setup
- Dashboard scripts use direct `fetch(${API}/...)` with Bearer token
- `API` derived from `import.meta.env.PUBLIC_API_BASE`

6. Data Flow & Processing

Webhook Flow

`Meta Instagram → POST /webhook/meta (Cloudflare Worker) → Verify hub.challenge + verify_token → Parse changes[].value (comment_id, media_id, text, from) → Enqueue job to COMMENT_QUEUE Worker processes via processCommentEvent(): → Lookup ig_account by ig_user_id (from payload) → Load media_configs[media_id] ?? account defaults → Filters: self-comment, dedup comment_id, keyword match → Comment reply on? POST /{comment_id}/replies → DM on? POST /{ig_user_id}/messages with recipient: { comment_id } → Insert comment_logs Rate cap: ~190 private replies/hour/account (queue overflow)`

Polling Fallback (Cron)

scheduled handler in api/src/index.ts runs every minute:

- Iterates igAccounts, lists recent comments via Meta Graph API
- Same processCommentEvent processor + dedup via commentLogs UNIQUE constraint

Data Persistence

- **Supabase:** Auth sessions only
- **Turso (libSQL):** All app data via Drizzle ORM — users, ig_accounts, media_configs, comment_logs, links, clicks, gate_waits, known_users, access_requests, byok_credentials, onboarding_state
- **Encryption:** Instagram tokens AES-GCM encrypted at rest (crypto.ts) using ENCRYPTION_KEY
- **Cache:** Cloudinary thumbnails (cloudinary.ts) — optional, falls back to Meta URLs

7. Database Schema (Turso / Drizzle)

Table	Key Columns
users	id, supabase_user_id UNIQUE, created_at
ig_accounts	ig_user_id UNIQUE, token_enc, token_expires, default_comment_reply, default_dm_text, dm_enabled, keyword
media_configs	media_id, comment_reply, dm_text, dm_enabled, keyword, (fallback to account defaults)
comment_logs	commentId, commenterId, action (comment_reply | private_reply), status, error, created_at
media_cache	account_id, media_id, caption, thumbnailUrl (Cloudinary URL), permalink
byok_credentials	ig_app_id_enc, ig_app_secret_enc, webhook_verify_token
onboarding_state	step, checks (JSON), updated_at

Source: packages/db/src/schema.ts — 15 tables total

8. Environment & Secrets

Cloudflare (wrangler.jsonc)

- TURSO_DATABASE_URL, TURSO_AUTH_TOKEN
- SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY (or anon + JWKS for JWT verify)

- META_APP_ID, META_APP_SECRET, WEBHOOK_VERIFY_TOKEN
- CLOUDINARY_URL (cloud_name, api_key, api_secret)
- Queue + KV + cron bindings

.env.example (apps/api/)

```
WEBHOOK_VERIFY_TOKEN=local_verify_token SUPABASE_URL=https://xyzcompany.supabase.co SUPABASE_ANON_KEY= TURSO_DATABASE_URL=libsql://your-db.turso.io TURSO_AUTH_TOKEN= META_APP_ID= META_APP_SECRET= CLOUDINARY_URL=cloudinary://key:secret@cloud API_BASE=http://localhost:8787 WEB_BASE=http://localhost:4321
```

.env.example (apps/web/)

```
PUBLIC_SUPABASE_URL=https://xyzcompany.supabase.co PUBLIC_SUPABASE_ANON_KEY= PUBLIC_API_BASE=http://localhost:8787
```

9. Cost Summary

Item	MVP	Scale Trigger
Cloudflare free tier	$0	Workers Paid $5/mo if limits exceeded
Supabase free	$0	paid plans after 50k MAU
Turso free	$0	paid row/s limits
Cloudinary free	$0	paid bandwidth
Meta API	$	—

Total	~$0	$5–25/mo realistic early scale

10. Summary Diagram

```
[User] → [Google OAuth] → [Supabase Auth] → [Hono API]
                                              │
                                              ├─ /health
                                              ├─ /me
                                              ├─ /accounts
                                              ├─ /webhook/meta ← Meta webhook
                                              ├─ /media
                                              ├─ /media-config
                                              ├─ /stats
                                              ├─ /byok
                                              └─ /onboarding
                                              └─[React/Astro UI]──────►
fetch(API_BASE + path, {headers: {Authorization: "Bearer " + token}})
```

Webhook → Queue → Worker → DB (Turso) → Meta API → Instagram